Этап 2. Концептуальное (инфологическое) проектирование

Что такое концептуальное проектирование

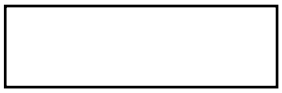
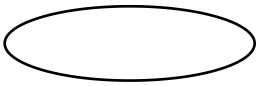
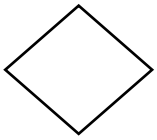
Концептуальное проектирование — это переход от текстового описания предметной области к графической модели данных, которая отражает сущности, их атрибуты и связи между ними. На этом этапе модель **не привязана к конкретной СУБД**: нет таблиц, колонок, типов данных, индексов. Есть только объекты реального мира и отношения между ними.

Цель — создать единую, непротиворечивую картину предметной области, которую понимают и заказчик, и разработчик. Для этого используют ER-модель.

ER-модель — это **язык общения** между аналитиком и программистом: заказчик видит знакомые понятия (читатель, книга, выдача), а разработчик — структуру, из которой потом определяется схема БД.

ER-модель: три кита

ER-модель (Entity-Relationship) строится на трёх типах элементов:

Элемент	Обозначение	Вид	Смысл
Сущность (Entity)	Прямоугольник		Класс объектов реального мира, о которых нужно хранить данные
Атрибут (Attribute)	Овал, связанный с сущностью линией		Свойство сущности или связи
Связь (Relationship)	Ромб, соединённый линиями с сущностями		Отношение между двумя (или более) сущностями

Сущность

1. Что это такое

Сущность — это множество однородных объектов, о которых система должна хранить информацию. Каждый отдельный объект — это **экземпляр** сущности. Например, сущность «Читатель» — это множество всех читателей библиотеки. Экземпляр — конкретный человек: Иван Петров, студент, билет №12345.

2. Как отличить сущность от атрибута

Частая ошибка — перепутать сущность и атрибут. Критерий:

- **Сущность** имеет собственные свойства, по которым нужно искать, фильтровать и устанавливать связи. У неё есть независимое существование в предметной области.
- **Атрибут** — это характеристика, которая описывает сущность, но сама по себе не является объектом учёта.

Пример: «Адрес» — это атрибут читателя, если адрес нужен только для справки. Но если система управляет доставкой книг на дом и адрес — это отдельный объект с городом, улицей, индексом, историей изменений, — тогда «Адрес» становится сущностью.

3. Типы сущностей

Тип	Описание	Пример
Сильная (независимая)	Существует сама по себе, имеет собственный ключ	Читатель, Книга, Зал
Слабая (зависимая)	Не может существовать без сильной сущности, ключ зависит от родителя	Этаж (без корпуса не имеет смысла), Экземпляр книги (без самой книги)

Слабая сущность обозначается двойным прямоугольником. Её ключ составной: собственный частичный ключ (дискриминант) + ключ родительской сущности.

4. Сколько сущностей выделять

Для учебного проекта — 5–10. Меньше пяти — модель слишком примитивная, не интересно проектировать. Больше десяти — теряется фокус, сложно нормализовать.

Критерий: если объект участвует в нескольких процессах, имеет несколько собственных атрибутов и связей — это сущность. Если упоминается только как характеристика одного объекта — атрибут.

Атрибут

1. Что это такое

Атрибут — это свойство сущности или связи, которое имеет значение для предметной области. Например, у читателя есть ФИО, дата рождения, номер билета. У книги — название, год издания, ISBN.

2. Ключевой атрибут

Ключевой атрибут (или комбинация атрибутов) **уникально идентифицирует** каждый экземпляр сущности. На ER-диаграмме ключевые атрибуты подчёркиваются.

Пример: «Номер читательского билета» — ключевой атрибут сущности «Читатель», потому что каждый билет уникален.

Если один атрибут не обеспечивает уникальность, ключ становится **составным**: комбинация двух и более атрибутов. Например, «Номер корпуса» + «Номер этажа» уникально идентифицирует этаж.

3. Типы атрибутов

Тип	Обозначение	Описание	Пример
Простой	Овал	Атомарное значение, неделимое	ФИО читателя
Составной	Овал с вложенными овалами	Состоит из частей, которые могут использоваться отдельно	ФИО = Фамилия + Имя + Отчество
Однозначный	Овал	Одно значение на экземпляр	Дата рождения
Многозначный	Двойной овал	Несколько значений на экземпляр	Телефоны читателя (их может быть несколько)
Производный	Пунктирный овал	Вычисляется из других атрибутов	Возраст = текущая дата − дата рождения
Ключевой	Подчёркнутый овал	Уникально идентифицирует экземпляр	Номер билета

На практике в учебных проектах чаще всего используют простые однозначные атрибуты. Многозначные и составные на этапе концептуального проектирования можно показать, но на этапе логического они будут разнесены по отдельным таблицам или колонкам.

4. Атрибуты связей

В нотации Чена связь тоже может иметь атрибуты. Например, связь «Читатель берёт Книгу» (M:N) может иметь атрибуты: дата выдачи, срок, дата возврата. Это атрибуты именно связи, а не сущностей, потому что они имеют смысл только в контексте конкретной выдачи конкретной книги конкретному читателю.

Позже, на этапе логического проектирования, M:N-связь с атрибутами превратится в промежуточную таблицу, и эти атрибуты станут её колонками.

Связь

1. Что это такое

Связь — это отношение между сущностями, которое имеет смысл в предметной области. Каждая связь имеет **имя** (глагол или словосочетание): «читает», «оформляет», «находится в».

Связь соединяет две сущности (бинарная связь) или более (n-арная связь). В учебных проектах чаще всего встречаются бинарные связи между двумя сущностями.

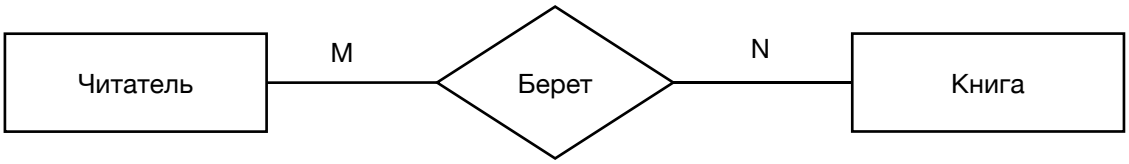
2. Степень связи (кардинальность)

Кардинальность показывает, **сколько экземпляров** одной сущности связано с экземпляром другой.

Обозначение	Название	Смысл	Пример
1:1	Один к одному	Один экземпляр A связан с одним экземпляром B, и наоборот	Читатель → Читательский билет
1:M	Один ко многим	Один экземпляр A связан с несколькими B, но каждый B — только с одним A	Зал → Стеллажи
M:N	Многие ко многим	Один экземпляр A связан с несколькими B, и один B — с несколькими A	Читатель → Книги

3. Как изображать кардинальность в нотации Чена

В нотации Чена кардинальность обозначается на линиях у ромба:



Здесь: Один читатель берёт много книг, одна книга выдаётся многим читателям (в разное время).

4. Типы кардинальностей с учётом обязательности

Кроме количества, важна **обязательность** связи:

Тип	Обозначение	Смысл	Пример
Обязательная	1 (без нуля)	Каждый экземпляр обязан участвовать	Каждый экземпляр книги находится в каком-то зале
Необязательная	0	Экземпляр может не участвовать	Читатель может не иметь активных выдач
Точно N	N	Ровно N экземпляров	Каждая выдача связана ровно с 1 читателем
От 0 до N	0..N	От нуля до нескольких	У читателя от 0 до 5 книг на руках

В нотации Чена обязательность иногда показывают двойной линией (обязательная) и одинарной (необязательная).

Как правильно определять тип связи

Алгоритм для двух сущностей

1. Возьми один экземпляр сущности A. Спроси: **с сколькими экземплярами B он связан?** Ответ: один или много?
2. Возьми один экземпляр сущности B. Спроси: **с сколькими экземплярами A он связан?** Ответ: один или много?
3. Комбинация ответов даёт тип связи.

Пример для «Читатель — Книга»:

- Один читатель берёт **много** книг → M
- Одну книгу (экземпляр) может взять **один** читатель (одновременно) → 1

Получается M:1 (много книг к одному читателю). Но если рассматривать не конкретный экземпляр, а всю историю выдач: одну книгу брали много разных читателей в разное время → M:N.

Контекст определяет тип связи. Нужно понимать, о каком временном горизонте идёт речь: «прямо сейчас» или «за всю историю».

Частая ошибка: 1:M вместо M:N

Если спросить «книга — читатель», кажется что один экземпляр книги у одного читателя → 1:1. Но это только в момент выдачи. За всё время книгу брали разные читатели, а один читатель брал разные книги → M:N.

Правильный вопрос: **«Может ли один экземпляр A быть связан с более чем одним экземпляром B за всё время существования системы?»** Если да — это не 1:1.

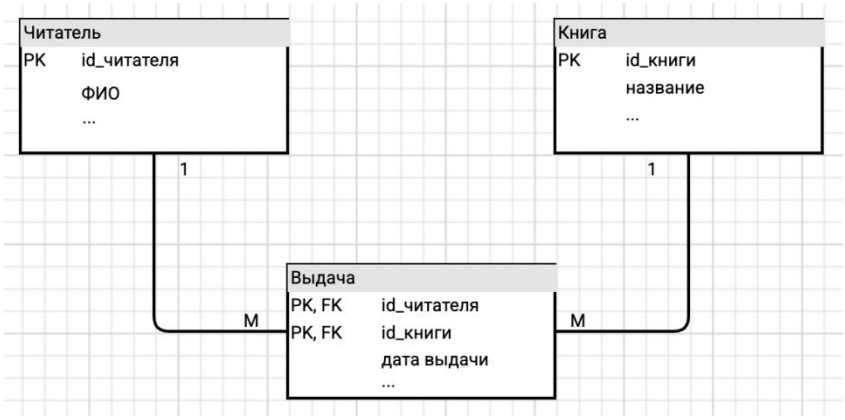
M:N-связи и почему они особые

M:N — единственный тип связи, который **нельзя напрямую реализовать в реляционной БД**. Реляционная модель поддерживает только внешние ключи (FK), а FK — это всегда ссылка от «многих» к «одному». Поэтому M:N-связь на этапе логического проектирования **всегда превращается в промежуточную таблицу** с двумя FK.

Пример:
Концептуальная модель:



Логическая модель:

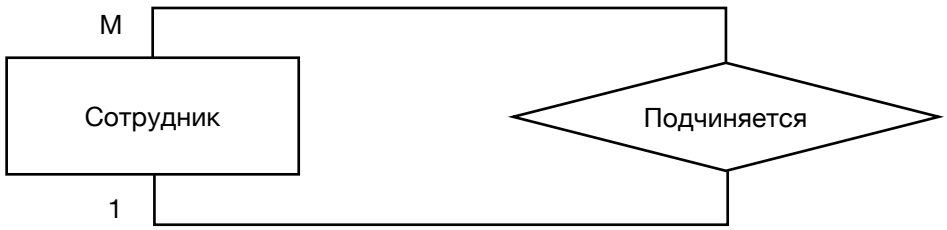


«Выдача» — это промежуточная таблица, которая реализует M:N-связь. Её первичный ключ — составной: (id_читателя, id_книги), или чаще — суррогатный id + уникальный индекс на пару FK.

Если у M:N-связи есть атрибуты (дата выдачи, срок, дата возврата), они переносятся в промежуточную таблицу. Это естественное место для них, потому что эти атрибуты описывают конкретную связь, а не сущность саму по себе.

Рекурсивные связи

Связь может соединять сущность **с самой собой**. Это называется рекурсивной связью.
Пример: сущность «Сотрудник» связана сама с собой через отношение «подчиняется»:



Один руководитель имеет много подчинённых, каждый подчинённый — одного руководителя.

На этапе логического проектирования это реализуется через FK в таблице «Сотрудник», ссылающийся на первичный ключ той же таблицы.

Нотация Чена: правила построения

1. Элементы

Элемент	Графическое обозначение
Сущность	Прямоугольник
Слабая сущность	Двойной прямоугольник
Атрибут	Овал, соединённый линией с сущностью
Ключевой атрибут	Подчёркнутый овал
Многозначный атрибут	Двойной овал
Производный атрибут	Пунктирный овал
Связь	Ромб, соединённый линиями с сущностями
Кардинальность	Числа (1, M, N) на линиях у ромба
Обязательная связь	Двойная линия
Слабая связь (идентифицирующая)	Двойной ромб

2. Порядок построения

1. **Выпиши все сущности** из этапа 1 (список сущностей и атрибутов).
2. **Размести прямоугольники** на холсте, группируя по смыслу.
3. **Добавь атрибуты** овалами к каждой сущности. Подчеркни ключевые.
4. **Найди связи** между сущностями. Назови каждую связь глаголом.
5. **Определи кардинальность** каждой связи по алгоритму из раздела 6.
6. **Проверь**: все ли сущности связаны? Нет ли изолированных? Нет ли связей без атрибутов, которые явно нужны? Не пропущены ли M:N?
7. **Добавь атрибуты к связям**, если они есть (например, «дата выдачи» на связи «берёт»).

3. Типичные ошибки

Ошибка	Пример	Почему плохо
Атрибут как сущность	«Город» как сущность, если нужен только для справки о читателе	Лишняя сущность усложняет модель
Сущность как атрибут	«Адрес» как атрибут, если нужно хранить историю адресов	Потеря данных
Не подчеркнут ключ	«Номер билета» без подчёркивания	Непонятно, как идентифицировать экземпляр
M:N отмечена как 1:M	«Читатель — Книга» как 1:M (один читатель — много книг)	Игнорирует, что книгу берут разные читатели
Связь без имени	Ромб без подписи	Непонятен смысл связи
Атрибуты связи привязаны к сущности	«Дата выдачи» висит на «Читателе»	Дата выдачи зависит от конкретной пары читатель-книга, а не от читателя
Избыточная связь	Две связи «Читатель — Книга» с разным смыслом, но одной кардинальностью	Обычно одна связь с атрибутом-типом решает задачу

Пример построения ER-модели (Библиотека)

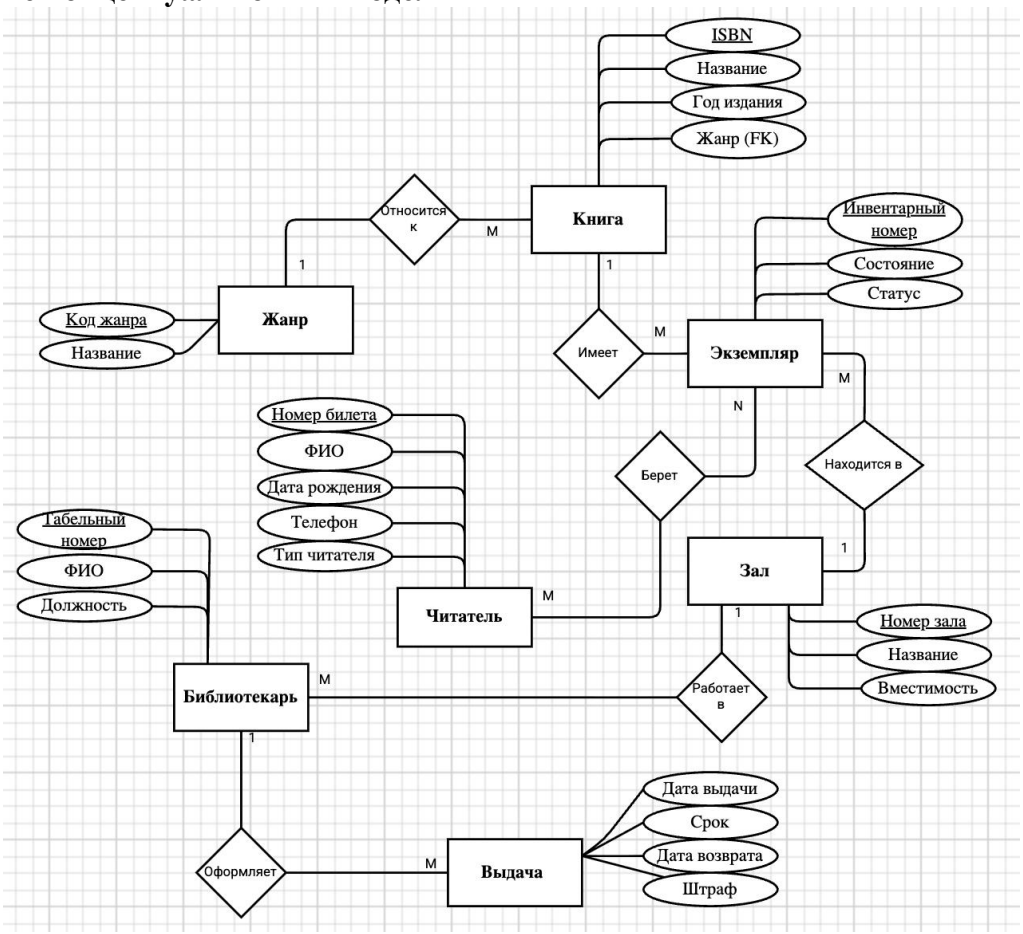
1. Сущности и атрибуты

Сущность	Атрибуты (ключевые подчёркнуты)
Читатель	Номер билета, ФИО, Дата рождения, Телефон, Тип читателя
Книга	ISBN, Название, Год издания, Жанр
Экземпляр	Инвентарный номер, Состояние, Статус
Зал	Номер зала, Название, Вместимость
Библиотекарь	Табельный номер, ФИО, Должность
Жанр	Код жанра, Название
Выдача	Дата выдачи, Срок, Дата возврата, Штраф

2. Связи и кардинальности

Связь	Сущности	Тип	Смысл
находится в	Экземпляр — Зал	M:1	Каждый экземпляр в одном зале, в зале много экземпляров
имеет	Экземпляр — Книга	M:1	Экземпляры одной книги (один ISBN → много инвентарных номеров)
относится к	Книга — Жанр	M:1	У книги один жанр (для упрощения), в жанре много книг
берёт	Читатель — Экземпляр	M:N	Читатель берёт много экземпляров, экземпляр берут много читателей (в разное время)
оформляет	Библиотекарь — Выдача	1:M	Библиотекарь оформляет много выдач, выдача оформляется одним библиотекарем
работает в	Библиотекарь — Зал	M:1	Библиотекарь закреплён за залом, в зале может работать несколько библиотекарей

3. **Построение концептуальной ER-модели**



4. Замечание о связи «берёт»

Связь «Читатель берёт Экземпляр» — M:N, потому что:

- один читатель за время существования системы берёт **много** экземпляров;
- один экземпляр за всё время **выдаётся многим** разным читателям (в разное время).

Эта связь имеет атрибуты (дата выдачи, срок, дата возврата, штраф), которые на этапе логического проектирования станут колонками промежуточной таблицы «Выдача».

5. Замечание о «Выдаче» как сущности vs связи

На концептуальном уровне «Выдача» может быть показана двояко:
- Вариант A: как связь с атрибутами
- Вариант B: как самостоятельная сущность

Если «Выдача» участвует в дополнительных связях (например, «Библиотекарь оформляет Выдачу»), то её удобнее показать как сущность. Тогда связи «Читатель — Выдача» (1:M) и «Экземпляр — Выдача» (1:M) заменяют M:N-связь.

Оба варианта допустимы. Вариант B чаще используется на практике, потому что в реальных системах у выдачи есть собственные связи и атрибуты, и удобнее работать с ней как с полноценной сущностью.

Проверка качества ER-модели

Критерий	Вопрос для проверки
Полнота	Все ли сущности из ТЗ отражены? Все ли процессы можно выполнить через эти сущности и связи?
Корректность связей	Правильно ли определены кардинальности? Проверены ли обе стороны каждой связи?
Отсутствие избыточности	Нет ли двух связей с одинаковым смыслом? Нет ли сущностей, которые можно объединить?
Атрибуты на месте	Атрибуты связи — на связи, а не на сущности? Ключевые атрибуты подчёркнуты?
M:N обработаны	Все ли M:N-связи отмечены и осознаны как будущие промежуточные таблицы?
Имена	Все ли связи названы? Все ли сущности и атрибуты имеют понятные имена?

Что должно быть в отчёте по этапу 2

Документ	Содержание
ER-диаграмма	Графическая схема в нотации Чена: сущности, атрибуты, связи, кардинальности
Описание связей	Таблица: связь → сущности → тип → смысл → обоснование типа
Описание слабых сущностей	Если есть — объяснение, почему сущность слабая и от чего зависит
Описание атрибутов связей	Если есть — какие атрибуты привязаны к связям и почему
Замечания о M:N	Явное указание, какие связи будут преобразованы в промежуточные таблицы

Связь с другими этапами

Следующий этап	Что берётся из ER-модели
Логическое проектирование (этап 3)	Каждая сущность → таблица, каждая M:N → промежуточная таблица, атрибуты → колонки, ключевые атрибуты → PK
Физическое проектирование (этап 4)	Типы связей и объёмы определяют индексы (FK → индекс), партиционирование (большие M:N-таблицы)
Реализация (этап 5)	CREATE TABLE строится из реляционной схемы, которая построена из ER-модели

Ошибки в ER-модели напрямую превращаются в ошибки в схеме БД. Например, если M:N ошибочно отмечена как 1:M, в реляционной схеме не будет промежуточной таблицы, и данные о множественных связях потеряются.